

Módulo 7 – Capítulo 10 – Sección 01

Métricas de tarea: tasa de completitud, precisión de pasos y eficiencia de herramientas

Nota de alcance: Este capítulo trata la evaluación de agentes en producción —medición sistemática del comportamiento sobre tráfico real, evolución de métricas en el tiempo, y ciclos de mejora continua basados en datos de producción—. El Capítulo 07 del módulo cubre el testing de agentes en desarrollo y CI/CD: validación pre-deploy con inputs sintéticos y golden trajectories en entornos controlados. Ambas dimensiones son necesarias y se complementan; no se sustituyen. Un agente que pasa todos los tests de Capítulo 07 puede exhibir comportamiento diferente en producción si la distribución de inputs de los usuarios reales difiere de los casos de prueba. La evaluación en producción que este capítulo examina cierra ese gap.

La pregunta más importante sobre un agente en producción no es "¿funciona?" sino "¿con qué frecuencia falla, en qué tipos de tareas, y por qué?". Las métricas de tarea son los indicadores cuantitativos que permiten responder esta pregunta de forma objetiva, comparable entre versiones del sistema, y trazable en el tiempo. Sin métricas formales, cualquier afirmación sobre la calidad del agente —"funciona bien", "ha mejorado con el último cambio de prompts", "es más confiable que la versión anterior"— es una opinión no verificable. Con métricas, esas afirmaciones se convierten en evidencia: "el TCR pasó del 78% al 85% entre la versión 2.1 y la 2.3, con una mejora especialmente notable en las tareas de categoría compleja".

La **Tasa de Completitud de Tareas** (Task Completion Rate, TCR) es la métrica primaria de efectividad: la fracción de tareas del conjunto de evaluación que el agente completa exitosamente según criterios explícitamente predefinidos. La definición de "completitud" debe especificarse antes de comenzar la medición —no inferirse de los resultados del agente— y debe ser evaluable de forma consistente sobre el conjunto de evaluación. Para tareas con criterios objetivos (¿el código generado pasa los tests?, ¿la información recuperada contiene la respuesta correcta?) la completitud puede evaluarse programáticamente. Para tareas con criterios subjetivos (¿la respuesta es suficientemente completa y útil?) se usa un LLM-judge

con rubric explícita. Calcular el TCR separadamente para diferentes categorías de tarea — simple, media, compleja; o por dominio de aplicación— es más informativo que una métrica global: un TCR global del 85% puede esconder un 95% en tareas simples y un 60% en tareas complejas.

La **Precisión de Pasos** (Step Accuracy) desciende al nivel de las decisiones individuales dentro del ciclo agéntico: ¿qué fracción de los pasos individuales en la trayectoria del agente son correctos? Un agente puede completar una tarea con TCR=1 pero con Step Accuracy baja si llegó al resultado correcto por un camino ineficiente o parcialmente incorrecto. Más revelador es el caso opuesto: un agente con Step Accuracy alta en los primeros pasos pero baja en un paso específico de la cadena puede tener un TCR bajo porque ese paso en particular es un cuello de botella sistémico. La Step Accuracy identifica el paso específico donde el agente más frecuentemente se desvía, lo que guía directamente la intervención de mejora.

La **Eficiencia de Herramientas** (Tool Call Efficiency) mide el ratio entre el número de herramientas que el agente invoca y el mínimo teórico necesario para completar la tarea. Un ratio de 1.0 indica que el agente usa exactamente las herramientas necesarias sin llamadas redundantes. Un ratio de 2.0 indica que el agente invoca el doble de herramientas de las necesarias: busca información que ya tiene en el contexto, repite la misma consulta con parámetros ligeramente diferentes, verifica resultados que no necesita verificar. La alta ineficiencia tiene consecuencias directas en latencia (más herramientas = más tiempo) y en costo (más tokens de observación = más tokens de input en las llamadas siguientes al LLM).

Métricas clave

- **Task Completion Rate (TCR):** porcentaje de tareas completadas exitosamente según criterios predefinidos (programáticos o LLM-judge); calcular por categoría de tarea (simple/media/compleja) para identificar dónde falla el agente; una variación de >3pp entre versiones es estadísticamente significativa con $n > 200$ tareas por categoría
- **Step Accuracy:** porcentaje de pasos individuales en la trayectoria que son correctos; evaluado contra golden trajectories anotadas o por LLM-judge por paso; identifica el punto específico de la cadena donde el agente se desvía sistemáticamente
- **Tool Call Efficiency:** ratio de llamadas realizadas sobre llamadas necesarias (ideal = 1.0, mayor que 1 indica ineficiencia); detecta búsquedas redundantes, verificaciones innecesarias, y bucles de corrección evitables
- **Token Efficiency:** tokens totales consumidos por tarea (input + output + observaciones de herramientas) dividido por la complejidad normalizada de la tarea; permite comparar

la eficiencia entre versiones y estimar el costo esperado por tarea

- **Mean Time to Complete (MTTC):** latencia media end-to-end por tarea (P50, P95, P99); monitorear su distribución para detectar outliers y degradaciones de performance entre versiones

***Nota del Arquitecto:** La elección del conjunto de evaluación es tan importante como la elección de las métricas. Un conjunto de evaluación curado manualmente con los casos más fáciles produce TCRs inflados que no predicen el desempeño en producción. El conjunto de evaluación debe incluir una muestra representativa de la distribución real de inputs de producción, incluyendo los casos que son difíciles o inusuales. El procedimiento correcto es construir el conjunto de evaluación inicial antes de comenzar la mejora del agente, de forma que las métricas sobre ese conjunto sean comparables entre versiones.*

Principio rector

Las métricas de tarea solo son útiles si se miden sobre distribuciones representativas de los casos de uso reales. Un TCR del 95% en un test set fácil y curado proporciona falsa confianza si el 50% de las tareas de producción son más complejas que las del test set. La construcción del conjunto de evaluación representativo es una inversión de ingeniería que se amortiza en todos los ciclos de mejora futuros.

Módulo 7 – Capítulo 10 – Sección 02

Métricas de razonamiento: coherencia del plan y calidad de las decisiones intermedias

Las métricas de tarea —TCR, Step Accuracy, Tool Efficiency— responden la pregunta de qué tan bien funciona el agente. Las métricas de razonamiento responden la pregunta más diagnóstica: por qué funciona así, y concretamente, dónde está el problema cuando algo falla. Esta distinción es la que diferencia el debugging de sistemas agénticos del debugging de software convencional. Un sistema de software que produce output incorrecto puede debuguearse trazando la ejecución del código hasta la línea que produce el error. Un agente que produce output incorrecto puede haberlo hecho por un error de razonamiento en el paso 3 que produjo un plan incorrecto, que llevó a una secuencia de acciones incorrectas, que convergió en un resultado incorrecto en el paso 15. Sin métricas de razonamiento, identificar el origen del error en esta cadena requiere revisar manualmente toda la trayectoria, lo que es inescalable cuando hay cientos de sesiones a analizar.

La **coherencia del plan** evalúa la calidad del razonamiento inicial del agente —cuando usa planificación explícita— antes de que comience la ejecución de acciones. Un plan coherente tiene tres propiedades verificables: es internamente consistente (los pasos no se contradicen entre sí), es suficiente para lograr el objetivo declarado (no le faltan pasos esenciales), y es libre de acciones redundantes o contradictorias. Un LLM-judge puede evaluar estas propiedades con alta confiabilidad dado que son verificaciones estructurales del texto del plan. El score de coherencia del plan (0-1 con justificación textual) identifica casos donde el agente parte con un plan defectuoso —la causa raíz más común de fallos de trayectoria completa— y la justificación del score proporciona información directamente accionable para la mejora del prompt de planificación.

El **grounding score** mide si el razonamiento del agente está anclado en evidencia del contexto —las observaciones acumuladas de herramientas, los datos proporcionados por el usuario, la información recuperada en pasos anteriores— o si el agente toma decisiones contradictorias a la evidencia disponible (alucinación de contexto). La alucinación de contexto es un patrón de fallo frecuente en tareas largas: el agente recupera información correcta

mediante una herramienta en el paso 5, pero en el paso 12 "recuerda" un valor diferente al que estaba en la observación del paso 5, y razona basándose en el valor incorrecto. Un grounding score bajo en los pasos finales de la trayectoria —cuando el contexto es más largo y el problema del lost-in-the-middle (Liu et al., 2023) es más pronunciado— es una señal para intervenir en la gestión de memoria del agente.

La **calidad de las decisiones intermedias** (Decision Quality) evalúa si, dado el estado completo del agente en el paso N —el historial acumulado, las observaciones disponibles, los pasos completados y pendientes—, la acción elegida es la más adecuada entre las alternativas disponibles. Esta métrica requiere un LLM-judge que recibe el estado completo del agente en el paso N y evalúa la acción elegida comparada con las alternativas que el agente podría haber elegido. La Decision Quality es especialmente valiosa para identificar patrones sistemáticos de decisiones subóptimas: ¿el agente tiende a buscar información que ya está en el contexto? ¿Tiende a sobrecompleja los planes cuando una aproximación directa es suficiente? ¿Tiende a abandonar herramientas antes de que hayan proveído resultados útiles?

La **faithfulness del razonamiento** (Reasoning Faithfulness) evalúa la consistencia entre el razonamiento visible del agente (el scratchpad de Chain-of-Thought) y la acción que el agente efectivamente toma. Un agente fiel es aquel cuyo razonamiento predice correctamente su siguiente acción: si el razonamiento concluye "necesito verificar el precio actual del producto en la base de datos", la siguiente acción debe ser una llamada a la herramienta de base de datos, no a la herramienta de búsqueda web. Razonamiento que lleva a la herramienta A pero el agente invoca la herramienta B indica un desacoplamiento entre el proceso de razonamiento que el usuario ve y el proceso de decisión que realmente ocurre internamente, lo que hace el sistema menos interpretable y más difícil de mejorar.

Aspectos técnicos

- **Plan coherence score:** LLM-judge evalúa el plan inicial del agente (cuando usa planificación explícita) contra tres criterios: consistencia interna, suficiencia para el objetivo, y ausencia de redundancias; score 0-1 con justificación textual; identifica planes defectuosos antes de que produzcan trayectorias completas incorrectas
- **Grounding score:** verifica si las decisiones del agente son consistentes con las observaciones de herramientas disponibles en el contexto; un grounding score bajo en pasos tardíos de trayectorias largas indica lost-in-the-middle y es señal para intervenir en la gestión de memoria

- **Decision quality:** LLM-judge que recibe el estado completo del agente en el paso N y evalúa la optimalidad de la acción elegida comparada con las alternativas disponibles; identifica patrones sistemáticos de decisiones subóptimas que no son visibles en las métricas de tarea
- **Observation utilization:** mide si el agente incorpora activamente las observaciones de herramientas en su razonamiento posterior; detecta el patrón de "ejecutar la misma herramienta con los mismos parámetros múltiples veces" como señal de baja utilización de observaciones
- **Reasoning faithfulness:** para agentes con razonamiento visible (CoT), verificar si el razonamiento predice correctamente la acción siguiente; razonamiento que lleva a herramienta A pero el agente invoca herramienta B indica unfaithful reasoning y reduce la interpretabilidad del sistema

Nota del Arquitecto: Las métricas de razonamiento son costosas de evaluar porque requieren un LLM-judge por cada paso de cada sesión evaluada, no solo por el resultado final. Para producción de alto volumen, evaluar el 100% de las sesiones con todas las métricas de razonamiento es prohibitivo en costo. La solución práctica es evaluar estas métricas solo sobre muestras: el 100% de las sesiones que fallaron ($TCR=0$), una muestra aleatoria del 5-10% de las sesiones exitosas, y el 100% de las sesiones que los usuarios marcaron con feedback negativo. Esta cobertura proporciona diagnóstico de la causa raíz de los fallos sin el costo de evaluar todas las sesiones.

Para recordar

Las métricas de razonamiento son el diagnóstico clínico del agente: mientras las métricas de tarea dicen si el resultado es correcto, las métricas de razonamiento dicen en qué paso específico del proceso de decisión está el problema, lo que guía directamente las intervenciones de mejora. Sin diagnóstico del proceso, la mejora del agente es un proceso de ensayo y error sin dirección sistemática.

Módulo 7 – Capítulo 10 – Sección 03

Benchmarks de agentes: SWE-bench, GAIA y AgentBench

La evaluación interna de un sistema agéntico —métricas propias sobre un test set del dominio específico— proporciona la señal más relevante para las decisiones de mejora del sistema. Los benchmarks públicos estandarizados responden una pregunta diferente pero igualmente valiosa: ¿cómo se compara el sistema con el estado del arte? Los benchmarks permiten comparaciones directas entre diferentes modelos base, diferentes frameworks, y diferentes configuraciones de agentes sobre conjuntos de tareas reproducibles y verificables por la comunidad. Sin benchmarks estandarizados, las afirmaciones sobre el desempeño relativo de diferentes sistemas son comparaciones de sistemas en diferentes dominios con diferentes criterios de evaluación, que no son directamente comparables.

SWE-bench (Software Engineering Benchmark, Princeton/Chicago, 2023) es el benchmark de mayor impacto en la evaluación de agentes de ingeniería de software. El conjunto de evaluación consiste en 2.294 issues reales de 12 repositorios Python de producción ampliamente usados —Django, Flask, Scikit-learn, SymPy, Matplotlib, entre otros— donde el agente recibe el texto del issue, el estado del repositorio en el momento en que el issue fue reportado, y el conjunto de tests asociados. La tarea es producir un patch de código que resuelva el issue y pase los tests del issue sin romper otros tests del repositorio. Esta estructura convierte la evaluación en completamente objetiva y automatizable: un sistema externo aplica el patch al repositorio y ejecuta los tests, sin intervención de evaluadores humanos.

SWE-bench es notoriamente difícil. Los primeros sistemas agénticos evaluados en 2023 alcanzaban tasas de resolución del 2-4%. Los mejores sistemas de 2024-2025 alcanzan tasas del 30-50% en **SWE-bench Verified** —una versión curada de 500 issues verificados manualmente como de alta calidad y no ambiguos— y del 20-35% en **SWE-bench Lite** (300 issues representativos). La evaluación en el conjunto completo de 2.294 issues es computacionalmente costosa (requiere ejecutar la suite de tests de cada repositorio para cada issue), por lo que la mayoría de los trabajos de investigación y los proveedores de modelos publican resultados en Lite o Verified. Los resultados de SWE-bench son directamente

comparables entre sistemas porque el conjunto de evaluación, los criterios de evaluación, y el proceso de evaluación son completamente públicos y reproducibles.

GAIA (General AI Assistants benchmark, Meta AI/HuggingFace, 2023) evalúa capacidades de asistente general con 466 preguntas distribuidas en tres niveles de dificultad creciente. El nivel 1 requiere capacidades de uso de una herramienta para responder preguntas factuales con pasos de razonamiento simples. El nivel 2 involucra tareas multi-step con 2 a 5 herramientas diferentes. El nivel 3 incluye tareas complejas que requieren más de 5 herramientas con razonamiento profundo y síntesis de información de múltiples fuentes. Las herramientas requeridas incluyen búsqueda web, procesamiento de archivos en múltiples formatos (PDF, Excel, imágenes, audio), y razonamiento matemático. Los mejores sistemas actuales alcanzan aproximadamente un 70% en nivel 1 y un 30% en nivel 3. Las preguntas de GAIA están diseñadas para ser respondibles por humanos en minutos, estableciendo un límite de referencia humano claro.

AgentBench (Tsinghua University, 2023) evalúa agentes en 8 entornos de naturaleza radicalmente diferente: OS (comandos bash en un sistema operativo Linux), DB (consultas SQL y MongoDB a bases de datos reales), KG (consultas SPARQL a grafos de conocimiento), Lateral Thinking (puzzles de pensamiento lateral), ALFWorld (navegación en entornos 3D textuales), WebShop (e-commerce simulado), Mind2Web (navegación web en sitios reales), y HouseholdThink (simulación de hogar). La métrica compuesta promedia el desempeño sobre todos los entornos, penalizando a los sistemas que son especialistas en un subconjunto de entornos pero débiles en otros. AgentBench es valioso porque evalúa la generalización de las capacidades agénticas a través de tipos de entornos muy diferentes.

Aspectos técnicos

- **SWE-bench:** 2.294 issues de repositorios Python de producción; evaluación objetiva automatizada (el patch pasa/no pasa los tests); versiones Lite (300) y Verified (500) para evaluación más rápida; los mejores sistemas 2024-2025 alcanzan 30-50% en Verified
- **SWE-bench Lite y Verified:** subconjuntos para evaluación más rápida y confiable; Verified es la versión curada manualmente para eliminar issues ambiguos; la mayoría de publicaciones reportan resultados en Lite o Verified por el costo computacional del conjunto completo
- **GAIA:** 466 preguntas en 3 niveles; nivel 1 (factual, 1 herramienta), nivel 2 (multi-step, 2-5 herramientas), nivel 3 (complejo, >5 herramientas); los mejores sistemas ~70% en nivel 1, ~30% en nivel 3; establece límite de referencia humana clara

- **AgentBench:** 8 entornos heterogéneos incluyendo OS, DB, KG, web, y simulaciones; métrica compuesta que promedia el desempeño sobre todos los entornos; penaliza la especialización excesiva en un subconjunto de entornos
- **Data contamination y benchmarks dinámicos:** los benchmarks públicos pueden estar presentes en los datos de entrenamiento de los modelos evaluados; evaluar sobre benchmarks privados o dinámicamente generados (que varían entre evaluaciones) es más indicativo del desempeño real en producción

***Nota del Arquitecto:** Los benchmarks públicos son necesarios para comparaciones de investigación y para calibrar el sistema contra el estado del arte, pero no son suficientes para tomar decisiones de despliegue en un dominio específico. Un sistema que score 50% en SWE-bench puede ser inadecuado para un dominio de análisis financiero que requiere habilidades completamente diferentes. La evaluación en benchmarks privados construidos con tareas reales del dominio de aplicación siempre debe preceder y fundamentar la decisión de despliegue.*

Principio rector

Los benchmarks públicos son el lenguaje común de la comunidad de investigación agéntica: permiten comparaciones reproducibles entre sistemas. Pero el benchmark más relevante para cualquier sistema de producción es el que está construido con las tareas reales de su dominio de aplicación: esa es la evaluación que predice el desempeño real con los usuarios reales.

Módulo 7 – Capítulo 10 – Sección 04

Evaluación humana: validación de trayectorias y resultados finales

Los evaluadores automáticos —criterios programáticos, LLM-judge con rubric, métricas de trayectoria— son instrumentos de medición escalables y reproducibles. Pero la escala y la reproducibilidad tienen un costo: los evaluadores automáticos son tan buenos como los criterios con los que fueron diseñados, y hay dimensiones de calidad del agente que son genuinamente difíciles o imposibles de formalizar con suficiente precisión para un evaluador automático. La corrección de un análisis legal requiere juicio sobre la aplicabilidad de la jurisprudencia al caso específico. La calidad de un plan de proyecto requiere experiencia en gestión de proyectos del dominio. La utilidad real de una respuesta de customer support requiere entender el contexto implícito del usuario que no está en el texto de la pregunta. Para estas dimensiones, la evaluación humana no es un lujo sino el único estándar de evaluación con validez suficiente.

La evaluación humana de agentes tiene dos niveles que sirven propósitos diferentes. La **evaluación del resultado final** —¿la respuesta del agente es correcta, completa y útil?— es más rápida de realizar y más fácil de escalar: el anotador recibe el input del usuario y la respuesta final del agente, sin ver la trayectoria de ejecución. Es el tipo de evaluación más apropiada para medir la satisfacción del usuario final y para comparar versiones del sistema de forma rápida. La **evaluación de la trayectoria** —¿la secuencia de pasos fue razonable y eficiente?— es más costosa en tiempo del anotador pero proporciona información diagnóstica de mayor resolución: el anotador ve cada paso del ciclo de razonamiento-acción, cada invocación de herramienta y su observación, y puede identificar exactamente en qué paso el agente tomó la decisión incorrecta y articular por qué fue incorrecta. Esta información es la más valiosa para guiar los ciclos de mejora del sistema.

Las **annotation guidelines** son el documento que convierte la evaluación humana de una actividad subjetiva en un proceso con resultados reproducibles entre diferentes anotadores. Las guidelines deben especificar: los criterios de evaluación (¿qué significa "correcto"? ¿qué significa "completo"?), una rubric con ejemplos concretos de respuestas de cada nivel de

puntuación (ejemplos de score 1, 2, 3, 4 y 5 para anclar el juicio subjetivo a referencias objetivas), los casos límite más frecuentes y cómo evaluarlos, y el procedimiento para casos donde los criterios son insuficientes para tomar una decisión. Sin annotation guidelines explícitas, diferentes anotadores evaluarán los mismos casos de forma diferente, produciendo datos de evaluación con ruido que no son útiles para comparar versiones del sistema.

El **Inter-Annotator Agreement (IAA)** mide la consistencia entre diferentes anotadores sobre los mismos casos. Las métricas estándar son el Cohen's Kappa (para dos anotadores) y el Krippendorff's Alpha (para múltiples anotadores). Un IAA inferior a 0.6 indica que los criterios de evaluación son demasiado ambiguos para producir juicios consistentes —los anotadores no entienden lo mismo por "respuesta correcta"— y que las annotation guidelines deben refinarse antes de continuar. Un IAA superior a 0.8 indica criterios bien definidos y anotadores bien calibrados. Medir el IAA al inicio de cada ciclo de evaluación, y especialmente cuando se incorporan nuevos anotadores al proceso, es parte del control de calidad de los datos de evaluación.

El **sampling estratificado** maximiza el valor de la evaluación humana por unidad de tiempo del anotador. En lugar de evaluar todos los outputs del agente de forma aleatoria, priorizar: el 100% de las sesiones donde los evaluadores automáticos dan scores extremadamente bajos (identificadas como fallos probables), el 100% de las sesiones donde los usuarios proporcionaron feedback negativo explícito, el 100% de las sesiones que alcanzaron `max_steps` (tareas que el agente no pudo completar), y un sample aleatorio del 5-10% de las sesiones con scores normales (para calibrar el LLM-judge contra el juicio humano). Esta estrategia proporciona cobertura completa de los casos problemáticos y una muestra representativa de los casos normales, sin requerir evaluación humana de todas las sesiones.

Aspectos técnicos

- **Annotation guidelines:** documentar criterios de evaluación con rubric de 5 puntos y ejemplos concretos para cada nivel; casos límite explícitamente documentados; procedimiento para casos donde los criterios son insuficientes; revisión periódica de las guidelines cuando surgen casos no previstos
- **Inter-Annotator Agreement (IAA):** medir con Cohen's Kappa (dos anotadores) o Krippendorff's Alpha (múltiples); $IAA < 0.6$ indica ambigüedad en los criterios que debe resolverse antes de continuar; $IAA > 0.8$ indica criterios bien definidos; medir al inicio de cada ciclo de evaluación

- **Evaluación de trayectoria paso a paso:** el anotador recibe el objetivo de la tarea, el historial completo de reasoning + tool calls + observations, y evalúa cada paso como: correcto / aceptable / incorrecto con justificación; los pasos más frecuentemente evaluados como incorrectos revelan los puntos de fallo sistémicos
- **Sampling estratificado:** 100% de las sesiones con scores automáticos extremos o feedback negativo explícito del usuario, y 5-10% de muestra aleatoria de sesiones normales para calibración; maximiza el valor informativo por unidad de tiempo del anotador
- **Human-AI evaluation pipeline:** evaluación automática para el volumen, evaluación humana para los casos de baja confianza del evaluador automático y para la calibración periódica; los casos donde el juicio humano difiere del automático son los más valiosos para mejorar el LLM-judge

Nota del Arquitecto: El error más común en evaluación humana de agentes es omitirla porque "tenemos un LLM-judge que ya evalúa automáticamente". El LLM-judge es una aproximación al juicio humano, no un sustituto: está calibrado contra ejemplos que el equipo proporcionó, que pueden no ser representativos del espacio completo de outputs del agente. La evaluación humana periódica —no continua, pero sí regular— es la única forma de verificar que el LLM-judge sigue estando calibrado correctamente a medida que el agente evoluciona y produce outputs en distribuciones que los ejemplos originales no cubrían.

Buena práctica

Implementar un feedback loop donde los resultados de la evaluación humana alimentan directamente la calibración del LLM-judge. Los casos donde el juicio humano difiere significativamente del juicio automático —en cualquier dirección— son los más informativos para identificar los límites del evaluador automático y mejorar su calibración con ejemplos adicionales. Este ciclo de mejora del evaluador es tan importante como el ciclo de mejora del agente.

Módulo 7 – Capítulo 10 – Sección 05

Mejora continua: aprender de los fallos del agente en producción

Los fallos del agente en producción tienen una propiedad que los convierte en la fuente de información más valiosa para mejorar el sistema: son casos reales. No son escenarios hipotéticos diseñados en el escritorio de un ingeniero, ni casos del test set curados para ser representativos; son instancias concretas donde un usuario real, con un objetivo real, proporcionó un input real que el agente real no pudo manejar satisfactoriamente. La información que contienen —el input exacto que causó el fallo, la trayectoria que siguió el agente, el output que produjo, las observaciones de las herramientas que recibió— es precisamente el tipo de dato que no puede generarse sintéticamente con la misma fidelidad. Los fallos de producción son el dataset de entrenamiento del próximo ciclo de mejora.

El **pipeline de mejora continua** basado en fallos de producción sigue un ciclo de cinco etapas. La primera es la **identificación de fallos**: recolectar señales que indican que el agente no satisfizo las expectativas del usuario. Las señales explícitas —botón de thumbs down, rating bajo, texto de feedback negativo— son las más directas pero las menos frecuentes (la mayoría de los usuarios que tienen una mala experiencia no dan feedback explícito). Las señales implícitas son más frecuentes: el usuario repite la misma pregunta reformulada en la misma sesión (señal de que la respuesta anterior no fue útil), el usuario abandona la sesión antes de obtener una respuesta completa, o métricas automáticas del sistema detectan que el agente alcanzó `max_steps` sin completar la tarea.

La segunda etapa es la **categorización de fallos**: clasificar cada fallo en una taxonomía predefinida que refleja los tipos de error que el agente puede cometer. Una taxonomía útil para agentes generales incluye: `reasoning_error` (el agente razonó incorrectamente dado el contexto disponible), `wrong_tool_selection` (el agente eligió la herramienta incorrecta para el paso en cuestión), `tool_execution_failure` (la herramienta falló y el agente no se recuperó adecuadamente), `context_window_overflow` (el contexto creció demasiado y el agente perdió información crítica), `hallucination` (el agente afirmó información que no está en el contexto ni en el conocimiento base verificable), `prompt_injection` (un input

adversarial redirigió el comportamiento del agente), y `user_misunderstanding` (el agente malinterpretó la intención del usuario). Para escalar la categorización a grandes volúmenes, un LLM-judge categorizador —entrenado sobre ejemplos anotados manualmente— puede clasificar automáticamente los fallos según esta taxonomía.

La tercera etapa es el **análisis de causa raíz** para categorías de fallo sistemáticas: cuando más del 5% de las sesiones exhiben el mismo tipo de fallo, escalar a un análisis formal. Revisar manualmente 20-50 casos de esa categoría, identificar el patrón común que aparece en todos o la mayoría de ellos, y formular una hipótesis de causa raíz que sea testeable: "el agente no usa la herramienta de búsqueda en base de datos cuando la pregunta menciona fechas, porque la descripción de la herramienta no menciona que puede filtrar por rango de fechas" es una hipótesis testeable; "el agente a veces falla en tareas complejas" no lo es.

La cuarta etapa es la **implementación y validación de la intervención**: diseñar la intervención específica que responde a la causa raíz identificada (cambio en la descripción de la herramienta, modificación del system prompt, adición de una nueva herramienta, ajuste de los ejemplos de few-shot), implementarla en un entorno de desarrollo, verificar que mejora el tipo de fallo en cuestión sobre el test set de casos de esa categoría, y validar que no introduce regresiones en otras categorías. Solo después de esta validación sobre el test set, desplegar la intervención al tráfico de producción mediante A/B testing con el 10-20% del tráfico, monitoreando el impacto sobre las métricas de producción antes de desplegarlo al 100%.

La quinta etapa es el **cierre del ciclo**: documentar la intervención (qué fue, por qué fue implementada, qué mejora produjo), añadir los casos de fallo que motivaron la intervención al test set de regresión (para que futuras versiones del agente sean evaluadas contra estos casos), y actualizar las annotation guidelines si la categoría de fallo reveló un criterio de evaluación que no estaba bien especificado. El conjunto de fallos categorizados y anotados que se acumula en este ciclo es el activo estratégico más valioso del equipo: contiene la memoria institucional de los errores que el sistema cometió y de las intervenciones que los resolvieron.

Aspectos técnicos

- **Señales de fallo multi-fuente**: combinar señales explícitas (thumbs down, rating bajo, feedback textual negativo) con señales implícitas (pregunta repetida en la misma sesión, abandono de sesión, sesiones que alcanzan max_steps); las señales implícitas tienen mayor volumen pero menor precisión que las explícitas

- **Taxonomía de fallos con LLM-categorizer:** un LLM-judge especializado en clasificar fallos según una taxonomía predefinida, entrenado con ejemplos anotados manualmente; escala la categorización a grandes volúmenes sin requerir anotación humana de cada fallo individual
- **Root Cause Analysis (RCA):** para categorías con >5% de presencia, revisión manual de 20-50 casos, identificación del patrón común, y formulación de hipótesis de causa raíz testeable; la hipótesis debe especificar qué cambio en el sistema debería reducir el fallo y qué métrica lo verificará
- **A/B testing de intervenciones:** desplegar cambios al 10-20% del tráfico antes de la adopción completa; comparar TCR, Step Accuracy, y distribución de categorías de fallo entre el grupo de control y el grupo de tratamiento durante al menos 7 días; revertir si el cambio no mejora o degrada otras métricas
- **Dataset de fallos como activo estratégico:** almacenar fallos categorizados y anotados en formato estructurado (task, expected_output, actual_output, failure_category, root_cause, intervention); este dataset es la base para fine-tuning futuro y para el test set de regresión de próximas versiones

Para recordar

Los fallos del agente en producción no son fracasos del sistema; son el dataset de entrenamiento del próximo ciclo de mejora. Cada fallo bien diagnosticado y categorizado es una oportunidad de hacer el agente más robusto ante esa clase de situación para todos los usuarios futuros. El equipo que trata los fallos de producción como datos —no como incidentes a resolver y olvidar— construye un sistema que mejora de forma acumulativa y sistemática con el tiempo.

Módulo 7 – Capítulo 10 – Sección 06

Cierre: medir agentes es la única forma de mejorarlos sistemáticamente

Sin un sistema de medición riguroso y continuo, cualquier mejora al agente es intuición, no ingeniería. Los cambios de prompt que "parecen mejores" pero no se miden contra un test set pueden mejorar un caso de uso mientras degradan diez que el ingeniero no estaba considerando. Los upgrades de modelo que "deberían funcionar mejor" pueden alterar comportamientos específicos del dominio que solo son detectables con tests de regresión sobre casos representativos del dominio real. Las nuevas herramientas que "amplían las capacidades" del agente pueden introducir conflictos de selección con herramientas existentes que reducen la precisión en tareas que ya funcionaban bien. La diferencia entre un equipo que mejora su agente de forma sistemática y uno que lo mejora de forma artesanal es la existencia de un sistema de evaluación que convierte cada cambio propuesto en una hipótesis verificable con datos.

La evaluación de agentes que este capítulo examina no es una actividad de final de proyecto sino una disciplina que se ejecuta continuamente, en cada cambio significativo del sistema: cuando cambia el modelo base, cuando se modifica el system prompt, cuando se añaden o modifican herramientas, cuando se cambia la configuración de memoria, y cuando el perfil de los usuarios o sus tareas evoluciona. Las métricas que se acumulan en el tiempo —TCR por categoría de tarea, Step Accuracy por tipo de decisión, distribución de categorías de fallo, costo medio por tarea— producen una visión de la evolución del sistema que ningún snapshot individual puede proporcionar. Un equipo que observa estas métricas durante meses puede detectar tendencias graduales de degradación que son invisibles en cualquier evaluación puntual.

Los cinco instrumentos de evaluación que este capítulo cubre —métricas de tarea, métricas de razonamiento, benchmarks públicos, evaluación humana, y mejora continua basada en fallos— son complementarios. Las métricas de tarea proporcionan el indicador de alto nivel de efectividad. Las métricas de razonamiento proporcionan el diagnóstico de causa raíz cuando las métricas de tarea degradan. Los benchmarks públicos posicionan el sistema en el contexto

del estado del arte. La evaluación humana proporciona el estándar de verdad para dimensiones que los evaluadores automáticos no capturan con suficiente fidelidad. Y el pipeline de mejora continua cierra el ciclo convirtiendo los datos de evaluación en cambios concretos del sistema que son validados antes de desplegarse.

La relación entre este capítulo y el resto del módulo de ingeniería de agentes es de cierre del ciclo completo: el Capítulo 01 definió qué es un agente y por qué es diferente de un chatbot; los Capítulos 02 a 06 examinaron cómo diseñarlo con razonamiento, herramientas, memoria, y arquitecturas multi-agente; los Capítulos 07 a 09 cubrieron cómo testearlo en desarrollo, asegurarlo ante adversarios, y desplegarlo en producción; este capítulo cierra el ciclo con la pregunta que retroalimenta todos los pasos anteriores: ¿cómo sabemos que funciona bien, y cómo lo mejoramos cuando no es así?

Epílogo del módulo: hacia los modelos locales e infraestructura propia

Este módulo ha construido el ciclo completo de ingeniería de agentes sobre la premisa implícita de que los modelos de lenguaje se acceden a través de APIs en la nube: OpenAI, Anthropic, Google. Esta premisa es la elección correcta para la mayoría de los equipos durante las etapas de prototipado, validación y despliegue inicial: las APIs de cloud eliminan la complejidad de infraestructura ML y permiten al equipo concentrarse en la lógica del agente. Pero una vez que los agentes están evaluados, en producción, y procesando volúmenes de tareas que generan costos de API significativos, emerge la siguiente pregunta de ingeniería: ¿cuándo y cómo tiene sentido ejecutar los agentes sobre modelos locales o infraestructura propia?

El **Módulo 8** responde precisamente esa pregunta. Examina cuándo el costo de las APIs de cloud justifica la inversión en infraestructura ML propia (GPUs on-premise o en cloud dedicado), cómo seleccionar y servir modelos locales (Llama, Mistral, Qwen) para diferentes componentes del sistema agéntico, qué capacidades del API cloud se pierden al pasar a modelos locales (context window, function calling, multimodal), y cómo el fine-tuning de modelos locales puede compensar parcialmente esas pérdidas en dominios específicos. La transición de cloud APIs a modelos locales no es una decisión binaria sino un espectro: es común que los sistemas de producción maduros usen modelos cloud para los componentes más críticos (el orquestador central, las decisiones de alto impacto) y modelos locales para los componentes de alta frecuencia y menor sensibilidad (clasificadores de intención, extractores de entidades, verificadores de formato). Dominar cuándo usar qué es el siguiente paso en la ingeniería de sistemas de IA a escala.

Para recordar

La diferencia entre un agente de investigación y un agente de producción es la misma que entre un experimento científico y un producto de ingeniería: ambos requieren hipótesis y resultados, pero solo el segundo requiere que esos resultados sean reproducibles, medibles y mejorables de forma sistemática y continua. Un equipo de AI Engineering que no mide sus agentes sistemáticamente no tiene ingeniería; tiene artesanía. Y la artesanía no escala.

"Without data, you're just another person with an opinion." — W. Edwards Deming, pionero de la gestión de calidad y el ciclo de mejora continua (PDCA); aplicado a sistemas agénticos: sin métricas de evaluación estructuradas y continuamente actualizadas, todas las decisiones de diseño del agente son opiniones —quizás opiniones bien informadas de ingenieros con experiencia—, pero opiniones al fin. La ingeniería de agentes comienza cuando esas opiniones se convierten en hipótesis verificables con datos.